CSU11021 Mid-Semester Test Cheat Sheet

2025-11-04

1 Two's Complement for Signed Numbers

- For 8 bits, the range is -128 to $+127$.
- Any value with a 1 in the leftmost bit is negative; any value with a 0 is positive.
- Conversion: flip every bit, then add 1 (converts between positive and negative signed values).

Example: What is -4 in 8-bit two's complement?

1. Find 4 in binary: 0000 0100
2. Flip all bits: 1111 1011
3. Add 1:

```
  1111 1011
+ 0000 0001
-----------
  1111 1100
```

2 Condition Code Flags

These flags are only modified by a CMP operation or an arithmetic operation with an s suffix. CMP acts as subtraction but does not write to a register – it only updates the flags.

- **C (Carry):** Set to 1 if the operation produces a carry out of the most significant bit (e.g. adding two 8-bit numbers where the result would require a 9th bit); otherwise 0.
- **Z (Zero):** Set to 1 if all bits of the result are zero; otherwise 0.
- **N (Negative):** Set to 1 if the most significant bit of the result is 1; otherwise 0.
- **V (oVerflow):** Set to 1 if both operands have the same sign but the result has a different sign; otherwise 0.

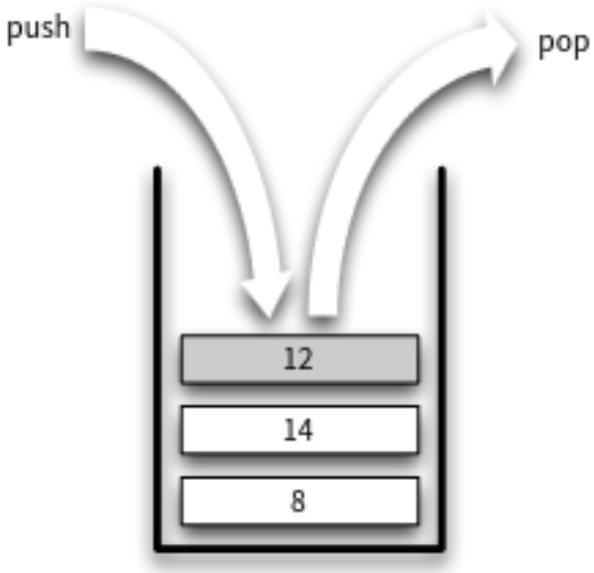
3 Arrays

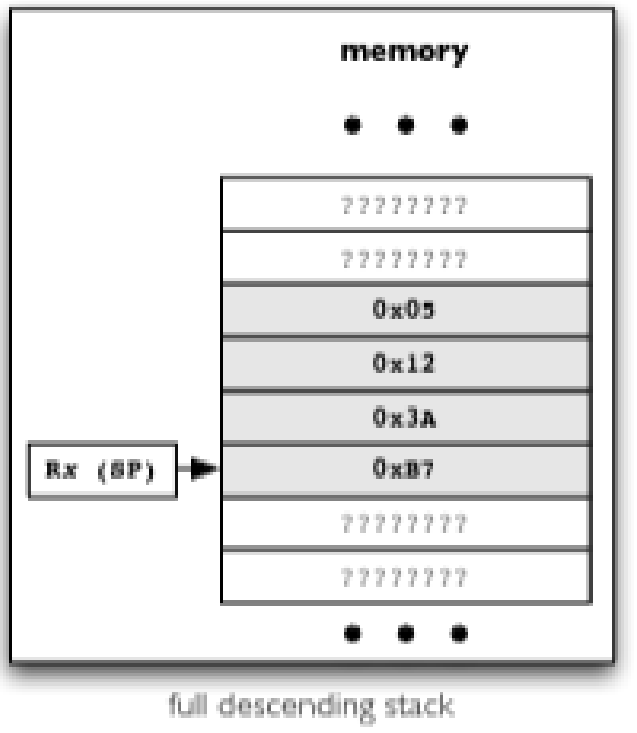
- Row-major ordering (row index comes first).
- To find the address offset for an element:

$$\begin{aligned} \text{index} &= (\text{row} \times \text{row\_size}) + \text{col} \\ \text{byte\_offset} &= \text{index} \times \text{elem\_size} \\ \text{address} &= \text{array\_base\_address} + \text{byte\_offset} \end{aligned}$$

4 How the Stack Works

- **PUSH** – Places items on top of the stack. Equivalent to: STR R0, [R13, #-4]!
- **POP** – Removes item from top of stack. Equivalent to: LDR R0, [R13], #4
- The stack grows downwards in ARM assembly (*full descending*).





full descending stack

Stack growth convention	push		pop	
	STM mode	stack-oriented synonym	LDM mode	stack-oriented synonym
full descending	STMDB	**STMFD or PUSH**	LDMIA	**LDMFD or POP**
empty ascending	STMIA	STMEA	LDMDB	LDMEA

Registers	Use
R0 … R3	Passing parameters to subroutines – avoid using for other variables – **corruptible (not saved/restored on stack)**
R4 … R12	Local variables within subroutines – **preserved (saved/ restored on stack)**
R13 (SP)	Stack Pointer – **preserved through proper use**
R14 (LR)	Link Register – **corrupted through subroutine call**
R15 (PC)	Program Counter

- LIFO (Last In, First Out).
- The highest-numbered register in curly braces is always pushed/popped first.
- Everything pushed onto the stack must be popped off.

5 Subroutines

- Parameters are passed using registers R0–R3.
  - If more parameters are needed, use the system stack:
    * Calling program pushes parameters onto the stack.
    * Subroutine accesses parameters relative to the stack pointer.
    * Calling program pops parameters off the stack after the subroutine returns.
  - R0–R3 should be assumed clobbered by the subroutine unless used to pass data in/out.
- If calling another subroutine from within a subroutine, **PUSH** the link register LR at the start.
- Subroutines must **PUSH** all registers they use onto the system stack at the *start* of the subroutine, and **POP** them back at the *end*.
- Subroutines can call themselves recursively (e.g. computing powers).
- All labels except main must be prefixed with .L.

6 Handy Reference

Hexadecimal Table

Hex	Decimal	Binary
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

MOV vs LDR

MOV can only store a limited range of values, whereas LDR does not have this limitation.

7 Definitions

- **bit** — 1 binary digit
- **byte** — 8 binary digits
- **halfword** — 2 bytes
- **word** — 4 bytes
- **LSL** — Logical Shift Left (equivalent to multiplying by 2^n)
- **LSR** — Logical Shift Right (equivalent to dividing by 2^n)
- **LDM / STM** — Load/Store Multiple. Requires word alignment (memoryAddress % 4 == 0)
- **LDMIA / STMIA** — IA = Increment After. Automatically increments the base address by the number of bytes read/written.

8 Efficiency Notes

- Every instruction takes at least one cycle.
- UDIV requires multiple cycles (dependent on the values involved).
- Each memory access costs one extra cycle. Relevant for: LDR, STR, LDM, STM, PUSH, POP.
- Reading words or halfwords across a word boundary costs *two* extra cycles instead of one.
- Taken branches incur a 2-cycle pipeline stall penalty.
  - Ensure you actually need the branch.
  - If-then-else constructs are expensive.
  - For conditional branches, optimise for the common case.

License

Copyright © 2026 Tommy Byrne

This work is free documentation: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This work is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this work. If not, see https://www.gnu.org/licenses/.

Source files are available at: https://github.com/mbdalpha/openNotes